Selenium Coding Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**23**
Exact duplicates removed (same question, different wording of punctuation or case)	**0**
Near-duplicates merged (same question asked in different words)	**2**
Unique questions in this PDF	**21**

Section 1: Critical (Must Practise)	5 questions
Section 2: High Priority	12 questions
Section 3: Medium Priority	4 questions

How to Use This Document

Tasks are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Practise) 5 questions	The most common live-coding asks in a Selenium automation round. Write each one by hand in your IDE until you can do it without notes.
High Priority 12 questions	Frequent follow-up tasks around test results, data and parallel execution. Practise the approach and the Java collections or Stream calls you would use.
Medium Priority 4 questions	Occasionally asked, usually for SDET-leaning roles. Know the approach even if you cannot write it perfectly.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
Framework & Page Object Coding	Can you write a clean LoginPage and a retry mechanism live, using best practices (private locators, reusable methods)?
Test Data, Config & API Data	Utility-class thinking: properties, Excel, random data, API response parsing, thread-safe data and caching.
Collections & Streams	Using List/Map/Queue/Streams on test results: group, filter, sort, find most-failing tests.
File Handling & Log Processing	Writing CSV results, comparing files and extracting unique errors from logs.

Note: this file was small (23 entries) with two merged pairs (properties-file reading and the test queue). The 'Coding tasks:' prefix was dropped. There is no Section 4 because every task here is realistic for a Selenium coding round.

Table of Contents

Section 1: Critical (Must Practise) (5 questions) . . . 5
Framework & Page Object Coding (2) . . . 5
Test Data, Config & API Data Handling (2) . . . 5
Collections & Streams on Test Results (1) . . . 6

Section 2: High Priority (12 questions) . . . 7
Test Data, Config & API Data Handling (4) . . . 7
Collections & Streams on Test Results (7) . . . 8
File Handling & Log Processing (1) . . . 9

Section 3: Medium Priority (4 questions) . . . 10
Test Data, Config & API Data Handling (1) . . . 10
Collections & Streams on Test Results (1) . . . 10
File Handling & Log Processing (2) . . . 10

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	Total
Framework & Page Object Coding	2 Qs, p.5	-	-	**2**
Test Data, Config & API Data Handling	2 Qs, p.5	4 Qs, p.7	1 Qs, p.10	**7**
Collections & Streams on Test Results	1 Qs, p.6	7 Qs, p.8	1 Qs, p.10	**9**
File Handling & Log Processing	-	1 Qs, p.9	2 Qs, p.10	**3**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Practise) (5 questions)

The most common live-coding asks in a Selenium automation round. Write each one by hand in your IDE until you can do it without notes.

Framework & Page Object Coding (2)

1. Explain the Page Object Model (POM). Write a sample LoginPage.java using best practices.

> **Answer:** POM models each page as a class: locators are private, actions are public methods, and tests contain no locators. Methods that move to another page return that page object. Add waits inside the page class and expose small readable actions.
>
> **Example:** LoginPage.login() returns HomePage; the test never touches a By locator.

```
public class LoginPage {
  private final WebDriver driver;
  private final WebDriverWait wait;
  private final By username = By.id("username");
  private final By password = By.id("password");
  private final By loginBtn = By.cssSelector("button[type='submit']");
  private final By error = By.cssSelector(".error-message");

  public LoginPage(WebDriver driver) {
    this.driver = driver;
    this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
  }
  public HomePage login(String user, String pass) {
    wait.until(ExpectedConditions.visibilityOfElementLocated(username)).sendKeys(user);
    driver.findElement(password).sendKeys(pass);
    driver.findElement(loginBtn).click();
    return new HomePage(driver);
  }
  public String getErrorMessage() {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(error)).getText();
  }
}
```

2. How do you run flaky test cases again? Show the retry code (e.g. IRetryAnalyzer).

> **Answer:** Implement IRetryAnalyzer with a counter and a maximum retry count, then attach it to a test or to all tests through a listener. Keep retries small (1-2) and log the first failure.
>
> **Example:** Rerun the payment test once if the gateway sandbox is slow, instead of failing the whole build.

```
public class RetryAnalyzer implements IRetryAnalyzer {
  private int count = 0;
  private static final int MAX_RETRY = 2;
  @Override public boolean retry(ITestResult result) {
    if (count < MAX_RETRY) { count++; return true; }
    return false;
  }
}
// per test: @Test(retryAnalyzer = RetryAnalyzer.class)
// for all tests: IAnnotationTransformer: annotation.setRetryAnalyzer(RetryAnalyzer.class);
```

Test Data, Config & API Data Handling (2)

3. Read properties from a config file (read key-value pairs from a properties file). **[asked x2 in your bank]**

> **Answer:** Load the file with the Properties class inside try-with-resources and read values with getProperty (with an optional default).
>
> **Example:** Read base.url, browser and timeout from config.properties.

```
static Properties readConfig(String path) throws IOException {
  Properties props = new Properties();
  try (FileInputStream in = new FileInputStream(path)) { props.load(in); }
  return props;
}
// Properties p = readConfig("src/test/resources/config.properties");
// String url = p.getProperty("base.url");
// String browser = p.getProperty("browser", "chrome");   // default value
```

4. How do you fetch test data from Excel?

> **Answer:** Use Apache POI: open the workbook, read the sheet, skip the header row and read each cell as text with DataFormatter; return Object[][] for a TestNG DataProvider.
>
> **Example:** Read username/password rows for the login DataProvider.

```
static Object[][] readExcel(String path, String sheetName) throws IOException {
  try (Workbook wb = WorkbookFactory.create(new File(path))) {
    Sheet sh = wb.getSheet(sheetName);
    DataFormatter fmt = new DataFormatter();
    int rows = sh.getLastRowNum(), cols = sh.getRow(0).getLastCellNum();
    Object[][] data = new Object[rows][cols];
    for (int i = 1; i <= rows; i++)
      for (int j = 0; j < cols; j++)
        data[i - 1][j] = fmt.formatCellValue(sh.getRow(i).getCell(j));
    return data;
  }
}
```

Collections & Streams on Test Results (1)

5. Find duplicate elements in a list.

> **Answer:** Keep a seen set and add an element to the duplicates set when add() returns false (O(n)). A stream version uses frequency counts.
>
> **Example:** Find duplicate product IDs in a bulk-upload list.

```
static <T> Set<T> findDuplicates(List<T> list) {
  Set<T> seen = new HashSet<>(), dup = new LinkedHashSet<>();
  for (T x : list) if (!seen.add(x)) dup.add(x);
  return dup;
}
// stream: list.stream().collect(Collectors.groupingBy(x -> x, Collectors.counting()))
//             .entrySet().stream().filter(e -> e.getValue() > 1).map(Map.Entry::getKey)...
```

Section 2: High Priority (12 questions)

Frequent follow-up tasks around test results, data and parallel execution. Practise the approach and the Java collections or Stream calls you would use.

Test Data, Config & API Data Handling (4)

6. How do you generate random test data dynamically?

> **Answer:** Use UUID, Random/ThreadLocalRandom for numbers and strings, or Datafaker for realistic names and addresses. Make values unique so parallel runs do not clash.
>
> **Example:** Create a new customer email for each registration test.

```
static String randomEmail() {
  return "user" + UUID.randomUUID().toString().substring(0, 8) + "@test.com";
}
static String randomString(int len) {
  String chars = "abcdefghijklmnopqrstuvwxyz";
  StringBuilder sb = new StringBuilder();
  Random r = new Random();
  for (int i = 0; i < len; i++) sb.append(chars.charAt(r.nextInt(chars.length())));
  return sb.toString();
}
static int randomInt(int min, int max) { return new Random().nextInt(max - min + 1) + min; }
```

7. How do you parse an API response and extract values?

> **Answer:** Send the request with Rest Assured, assert the status code and use jsonPath() to read single values, numbers and lists (or Jackson to map the body to a POJO).
>
> **Example:** Call GET /products/101 and verify name, price and the list of SKUs.

```
Response res = RestAssured.given().baseUri("https://api.shop.com").get("/products/101");
Assert.assertEquals(res.getStatusCode(), 200);
JsonPath json = res.jsonPath();
String name = json.getString("name");
double price = json.getDouble("price");
List<String> skus = json.getList("variants.sku");
```

8. How do you store test-specific data in thread-safe mode (ThreadLocal)?

> **Answer:** Store data in a ThreadLocal<Map> so each parallel thread gets its own copy; clear it after each test to avoid leaks.
>
> **Example:** Each parallel checkout test keeps its own orderId and user.

```
public class TestDataContext {
  private static final ThreadLocal<Map<String, String>> data =
      ThreadLocal.withInitial(HashMap::new);
  public static void set(String key, String value) { data.get().put(key, value); }
  public static String get(String key) { return data.get().get(key); }
  public static void clear() { data.remove(); }   // call in @AfterMethod
}
```

9. How do you merge two environment property sets into one?

> **Answer:** Create a new Properties (or Map) and putAll the base set first, then the environment set so environment values override the base values.
>
> **Example:** common.properties provides defaults and qa.properties overrides the base URL.

```
static Properties merge(Properties base, Properties env) {
  Properties merged = new Properties();
  merged.putAll(base);
  merged.putAll(env);   // env values override base values
  return merged;
}
```

Collections & Streams on Test Results (7)

10. How do you group test cases by module using Java Streams?

> **Answer:** Use stream().collect(Collectors.groupingBy(...)) on the module field. Add a downstream counting() to get the number per module.
>
> **Example:** Group results into Cart, Payment and Search modules.

```
static class TestResult {
  String id, module, status; long timeMs;
  TestResult(String id, String module, String status, long timeMs) {
    this.id = id; this.module = module; this.status = status; this.timeMs = timeMs;
  }
}
static Map<String, List<TestResult>> byModule(List<TestResult> results) {
  return results.stream().collect(Collectors.groupingBy(r -> r.module));
}
```

11. How do you check whether all test cases passed using Streams?

> **Answer:** Use allMatch() with the status check (it stops at the first failure).
>
> **Example:** Mark the smoke build green only if every test passed.

```
static boolean allPassed(List<TestResult> results) {   // TestResult as in Q10
  return results.stream().allMatch(r -> r.status.equals("PASS"));
}
```

12. How do you remove failed test cases from a result list?

> **Answer:** Use removeIf on a modifiable list, or filter into a new list to keep the original.
>
> **Example:** Keep only passed tests to calculate the average execution time.

```
results.removeIf(r -> r.status.equals("FAIL"));   // modifies the list
List<TestResult> passed = results.stream()
    .filter(r -> !r.status.equals("FAIL")).collect(Collectors.toList());
```

13. How do you map test case IDs with execution time dynamically?

> **Answer:** Use Collectors.toMap with id as key and time as value; add a merge function to handle duplicate ids.
>
> **Example:** Map each test id to its time to find the slowest tests.

```
static Map<String, Long> idToTime(List<TestResult> results) {
  return results.stream().collect(
      Collectors.toMap(r -> r.id, r -> r.timeMs, (a, b) -> a + b));   // merge duplicates
}
```

14. How do you find the most frequently failing test case?

> **Answer:** Filter failed results, group by test id with counting(), then take the entry with the highest count.
>
> **Example:** Across 10 nightly runs, find the checkout test that failed most often.

```
static String mostFailing(List<TestResult> runs) {
  return runs.stream().filter(r -> r.status.equals("FAIL"))
      .collect(Collectors.groupingBy(r -> r.id, Collectors.counting()))
      .entrySet().stream().max(Map.Entry.comparingByValue())
      .map(Map.Entry::getKey).orElse(null);
}
```

15. How do you sort test cases based on execution time dynamically?

> **Answer:** Use a Comparator on timeMs; reversed() gives descending order.
>
> **Example:** List the 5 slowest tests to target for optimisation.

```
results.sort(Comparator.comparingLong(r -> r.timeMs));                    // ascending
results.sort(Comparator.comparingLong((TestResult r) -> r.timeMs).reversed()); // descending
```

16. How do you use a Queue in automation scenarios (e.g. simulate a test queue for parallel execution)?

[asked x2 in your bank]

> **Answer:** Put test names in a thread-safe queue (ConcurrentLinkedQueue/BlockingQueue); worker threads poll() until it is empty, so each test runs once across threads.
>
> **Example:** Three threads pick the next test from a shared list of regression tests.

```
static void runQueue(List<String> tests, int threads) throws InterruptedException {
  Queue<String> queue = new ConcurrentLinkedQueue<>(tests);
  ExecutorService pool = Executors.newFixedThreadPool(threads);
  for (int i = 0; i < threads; i++)
    pool.submit(() -> {
      String t;
      while ((t = queue.poll()) != null)
        System.out.println(Thread.currentThread().getName() + " runs " + t);
    });
  pool.shutdown();
  pool.awaitTermination(1, TimeUnit.MINUTES);
}
```

File Handling & Log Processing (1)

17. How do you write test results to a CSV file?

> **Answer:** Write a header line and one line per result with a PrintWriter or Files.write inside try-with-resources; quote values that contain commas or use OpenCSV.
>
> **Example:** Export the nightly results to results.csv for the dashboard.

```
static void writeCsv(String path, List<TestResult> results) throws IOException {
  try (PrintWriter pw = new PrintWriter(new FileWriter(path))) {
    pw.println("TestId,Module,Status,TimeMs");
    for (TestResult r : results)
      pw.println(r.id + "," + r.module + "," + r.status + "," + r.timeMs);
  }
}
```

Section 3: Medium Priority (4 questions)

Occasionally asked, usually for SDET-leaning roles. Know the approach even if you cannot write it perfectly.

Test Data, Config & API Data Handling (1)

18. How do you cache reusable data for faster test execution?

> **Answer:** Keep reusable data in a ConcurrentHashMap and fetch it with computeIfAbsent so it is created once and shared safely.
>
> **Example:** Log in through the API once per user and reuse the token in all tests.

```
class DataCache {
  private static final Map<String, String> cache = new ConcurrentHashMap<>();
  static String getToken(String user) {
    return cache.computeIfAbsent(user, u -> fetchToken(u));   // API login only once
  }
  private static String fetchToken(String user) { return "token-" + user; }
}
```

Collections & Streams on Test Results (1)

19. How do you maintain browser session history in automation?

> **Answer:** The browser keeps its own history (navigate().back()/forward()). To track it in the framework, push each visited URL on a Deque and read the previous entry when needed; also log driver.getCurrentUrl() after navigation.
>
> **Example:** Check the user returns to the product page after pressing Back from the cart.

```
Deque<String> history = new ArrayDeque<>();
void visit(WebDriver d, String url) { d.get(url); history.push(url); }
String previous() { history.pop(); return history.peek(); }
```

File Handling & Log Processing (2)

20. How do you compare two text files and highlight the differences?

> **Answer:** Read both files into lists and compare line by line up to the longer size; print the line number and the two different lines.
>
> **Example:** Compare expected and downloaded invoice text files.

```
static void diff(String f1, String f2) throws IOException {
  List<String> a = Files.readAllLines(Paths.get(f1)), b = Files.readAllLines(Paths.get(f2));
  for (int i = 0; i < Math.max(a.size(), b.size()); i++) {
    String x = i < a.size() ? a.get(i) : "<missing>";
    String y = i < b.size() ? b.get(i) : "<missing>";
    if (!x.equals(y)) System.out.println("Line " + (i + 1) + ": " + x + " | " + y);
  }
}
```

21. How do you extract unique error messages from a large log file?

> **Answer:** Stream the file lazily with Files.lines (no full load into memory), keep lines containing ERROR, extract the message and collect it in a LinkedHashSet to remove duplicates.
>
> **Example:** Find the distinct errors in a 1 GB nightly server log.

```
static Set<String> uniqueErrors(String path) throws IOException {
  try (Stream<String> lines = Files.lines(Paths.get(path))) {
    return lines.filter(l -> l.contains("ERROR"))
        .map(l -> l.substring(l.indexOf("ERROR") + 5).trim())
        .collect(Collectors.toCollection(LinkedHashSet::new));
  }
}
```